Real-Time 4D Gaussian Splatting Acceleration via Motion-Aware Adaptive Pipeline and Foveated Rendering

Anonymous Author(s)

Abstract

A clear and well-documented LaTeX document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the "acmart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

Keywords: 4D Gaussian Splatting, Hardware Acceleration, Foveated Rendering, VR/AR, Deformation Field

ACM Reference Format:
Anonymous Author(s). 2018. Real-Time 4D Gaussian Splatting Acceleration via Motion-Aware Adaptive Pipeline and Foveated Rendering. In . ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The proliferation of edge rendering devices in augmented and virtual reality (AR/VR) necessitates the simulation of highly realistic environments. While static scene representations fail to capture the complexity of the real world, dynamic scenes introduce a temporal dimension that enables more comprehensive modeling of physical spaces. Recent advances in dynamic scene representation and rendering introduce new paradigms such as Neural Radiance Fields (NeRFs) and 4D Gaussian Splatting (4DGS). Compared to implicit and slower NeRFs, 4DGS emerges as a compelling solution due to its explicit geometric representation, differentiable mathematical properties, and superior rendering speed.

Despite these advantages, current 4DGS implementations struggle to achieve 20 frames per second (fps) on resource-constrained edge platforms, such as the Nvidia Jetson AGX Orin. This performance falls significantly short of the 60 Hz perception threshold required by the human visual system (HVS), making it inadequate for immersive edge applications. Through an in-depth analysis of the rendering pipeline, we observe that the deformation and rasterization stages consume the majority of execution time due to substantial redundant computations. Existing research on Gaussian Splatting accelerators primarily focuses on static scenes, thereby neglecting the analysis and optimization of deformation calculations specific to dynamic environments. Furthermore, although foveated rendering techniques leveraging HVS characteristics are essential for reducing rasterization overhead, they often overlook the unique features introduced by dynamic scenes, failing to achieve joint spatiotemporal acceleration. Additionally, direct mapping of 4DGS onto multi-core systems leads to severe tile load imbalance, which limits overall hardware utilization. To mitigate these inefficiencies, we propose a hardware and software co-design approach that enables 4DGS to achieve high frame rates under the strict power budgets of edge scenarios. Our main contributions are as follows:

- We provide a comprehensive profiling of the 4DGS rendering pipeline to expose critical computational bottlenecks and redundant operations.
- We propose a post-training motion-aware tagging method and a restructured dataflow that classifies Gaussian primitives based on temporal deformation to selectively bypass unnecessary calculations.
- We implement a spatio-temporal foveated rendering scheme that exploits both dynamic scene characteristics and human visual properties without compromising perceived quality.
- We design a dedicated hardware architecture and a workload-balancing scheduling scheme to optimize hardware utilization and memory access patterns. Our co-designed solution achieves a speedup of xxx× compared to the Jetson AGX Orin.

2 Background

2.1 4D Gaussian Splatting

Based on the point cloud of a static scene, 3D Gaussian splatting (3DGS) [7] trains a set of Gaussians, whose parameters include mean position $\mu \in \mathbb{R}^3$, covariance Σ, base opacity

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

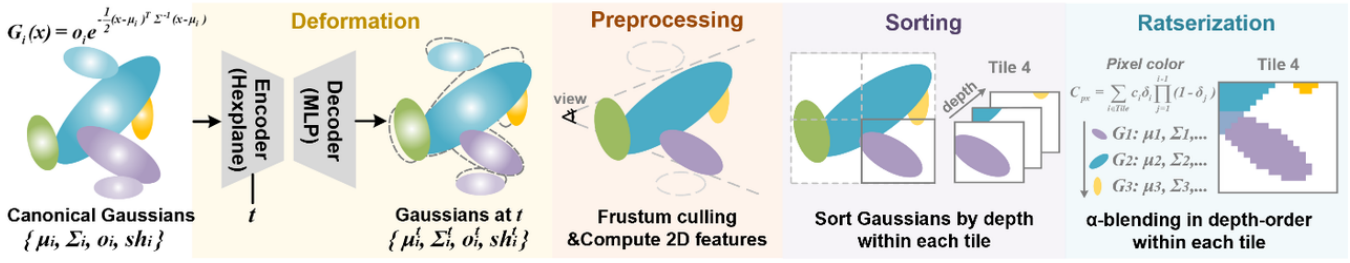

Figure 1. 4DGS rendering pipeline

o, and spherical harmonics (SH) coefficients sh. By introducing a temporal dimension, 4DGS extends the representational capacity of 3DGS from static environments to dynamic scenes. While 3DGS utilizes a set of time-independent Gaussian parameters, 4DGS formulates the parameters as a function of time. This methodology establishes a baseline static scene composed of canonical Gaussians and employs a continuous temporal function, often implemented as Gaussian distribution[13, 27] or neural network [5, 19, 22, 26], to model the deformation of each individual Gaussian across different time steps. Deformation fields constructed with neural networks offer superior representation capacity, which enables the precise modeling of complex dynamic scenes. To represent a dynamic scene, a dedicated set of 4D assets must be trained, encompassing both the canonical 3D Gaussians and the learned weights of the deformation field network. Once training concludes, these assets are deployed for rendering(i.e. inference). Given a specific viewpoint and a target time step t, the 4D representation collapses into an instantaneous static 3D model. The final 2D image is subsequently generated by accumulating the opacity and color attributes of the projected Gaussians for each pixel.

The complete rendering pipeline executes through four sequential stages as Fig. 1:

Deformation. Each frame designates a camera view and a time step t. A deformation field, which is usually a Nueral Network $\mathcal{N}$, predicts the required geometric adjustments. For a canonical Gaussian at position μ, the deformation offsets at t are computed as:

$$\Delta\mu, \Delta\Sigma, \Delta o, \Delta sh = \mathcal{N}(\mu, t)$$

These offsets are applied to the canonical properties to output the updated Gaussian parameters for the current time step t.

Preprocessing. Updated parameters are utilized to identify and cull Gaussians located outside the frustum. For the remaining visible primitives, their 2D screen-space features are computed, including projected center coordinates and covariance matrices. The image plane is then partitioned into uniform tiles, and a list of intersecting Gaussians is generated for each tile.

Sorting. These per-tile Gaussian lists are sorted based on the depth of the Gaussians to ensure correct visibility evaluation.

Rasterization. The color C of every pixel $\mathbf{x}$ within each tile is computed by performing iterative α-blending from front to back according to the volume rendering equation:

$$C = \sum_{i=1}^{N} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

where c_i is the view-dependent color derived from the SH coefficients, and the final evaluated opacity α_i for the pixel is determined by the 2D Gaussian distribution:

$$\alpha_i = o_i \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_i')^T (\Sigma_i')^{-1} (\mathbf{x} - \mu_i')\right)$$

This accumulation of opacity and color continues until the pixel reaches saturation ($\sum \alpha_i \approx 1$), successfully mapping the continuous Gaussian distributions into a discrete pixel space.

2.2 Gaussian Splatting Hardware Acceleration

A series of ASIC designs currently exist to accelerate the rendering of static scenes using 3DGS [6, 8, 15, 18, 20, 21, 24, 28]. GSCore[8] identifies memory bandwidth and rasterization latency as the primary bottlenecks within the rendering pipeline, proposing hierarchical sorting and fine-grained sub-tile skipping. Architectures such as GS-TG[6] and STREAMINGGS[28] introduce tile grouping and voxel-based streaming techniques to optimize memory access patterns. Neo[15] proposes a reuse and update sorting mechanism to minimize the overhead associated with depth sorting. Furthermore, hardware designs including GCC[18] and GSAcc[24] explore cross-stage conditional processing and depth prediction to eliminate redundant computations during the rasterization stage.

However, representing dynamic environments with 4DGS introduces a temporal dimension that necessitates deformation field computations for the Gaussian primitives. This requirement significantly amplifies the computational workload and memory traffic [4]. These emerging challenges remain underexplored in current hardware research. For

example, [25] mitigates preprocessing memory bottlenecks and pixel-level computational redundancy through quadratic interpolation and recursive computation reuse. Nevertheless, this approach neglects to exploit the intrinsic motion trends of the Gaussian primitives to prune deformation and preprocessing operations, and it overlooks the critical issue of inter-tile workload imbalance. Additionally, existing hardware solutions fail to incorporate foveated rendering schemes that leverage the physiological characteristics of HVS. Consequently, 4DGS retains substantial potential for further acceleration.

2.3 Foveated Rendering

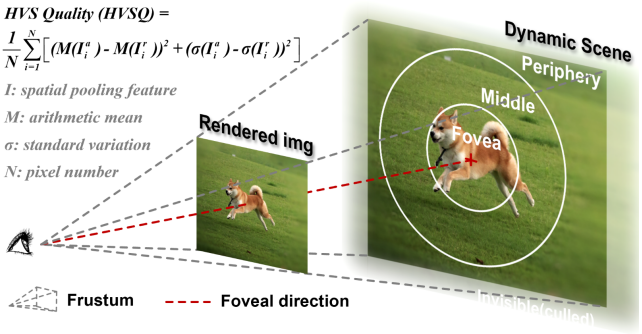

Figure 2. Foveated Rendering

The eye-tracking technology in VR/AR enables the practical implementation of foveated rendering (FR)[17], a technique that exploits the physiological characteristics of the human visual system (HVS). [3] proposed a mathematical model describing the linear degradation of visual acuity to illustrate the natural decline in peripheral vision. They designed a three-tier nested architecture that partitions the rendering frame into a central foveal region, a transition layer, and a peripheral zone based on the eccentricity from the user gaze point, as Fig 2. Beyond traditional polygon-based pipelines, researchers nowadays explore applying FR to emerging view synthesis algorithms including Gaussian splatting [1, 2]. These methods typically employ multi-resolution tile rasterization or adaptive mesh refinement to alleviate the computational demands of Gaussian primitives in peripheral visual areas. However, FR is rarely integrated with dynamic scene rendering.

Although FR introduces blur in peripheral areas, such effects remain imperceptible to HVS. Consequently, conventional visual quality metrics such as Peak Signal-to-Noise Ratio (PSNR) or Structural Similarity Index (SSIM) are unsuitable for evaluating FR as they fail to account for eccentricity-dependent acuity loss. To model the similarity between the reference and the altered images more scientifically, [10, 12] introduces HVS Quality (HVSQ) as Fig 2. This metric references a computational model of HVS and calculates spatial pooling features across different regions at varying granularities according to eccentricity, thereby simulating the feature extraction process in early human visual processing.

3 Motivation & Characterization

We comprehensively profile the 4DGS rendering [22] on the N3DV[9] and HyperNeRF[16] datasets with Jetson AGX Orin. This rigorous analysis exposes multiple performance bottlenecks and progressively yields several critical observations that motivate our proposed optimization and architectural design.

3.1 Latency Breakdown

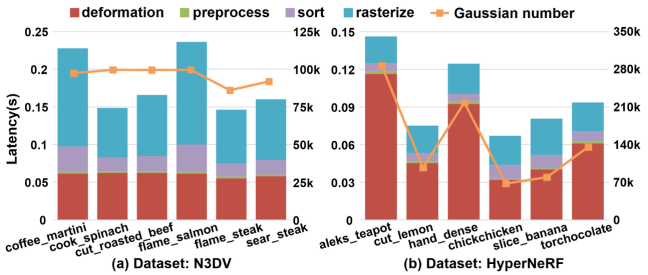

Figure 3. Latency breakdown of 4 stages

Observation 1*: Deformation and rasterization account for the majority of latency, which should be primarily optimized.* As Fig. 3, execution time profiling using NVIDIA Nsight Compute reveals that the deformation and rasterization stages dominate the rendering latency. The deformation computation accounts for approximately 30% to over 50% of the total execution time. Furthermore, scenes with a higher number of Gaussian primitives exhibit a proportionally larger deformation latency. This indicates that as the demand for complex scene modeling drives up the Gaussian count, the deformation overhead becomes increasingly critical, rendering existing architectures designed solely for 3DGS ineffective. Concurrently, the rasterization stage consumes a substantial portion of the execution time, presenting significant acceleration potential.

3.2 Gaussian Motion Analysis

Observation 2*: Considerable Gaussians are almost static, whose deformation computations are unnecessary.* Fig. 4 demonstrates the distribution of the maximum coordinate deformation across the entire time span for all Gaussian primitives, which indicates that vast amount of Gaussians remain nearly static. For example, Fig. 4a show that 50% Gaussians have a deformation amplitude ever less than 0.0048, which is tiny compared to the overall scene scale, meaning their deformation values approach 0 at any given time step. Bypassing these redundant computations yields substantial reductions in both latency and energy consumption without visual perception.

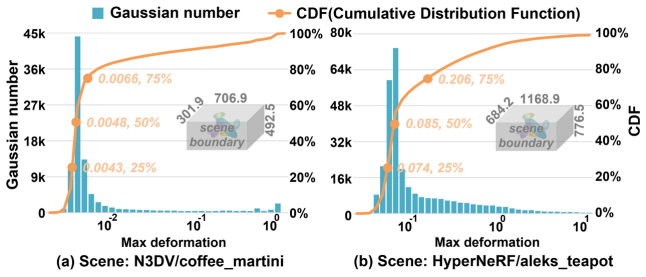

Figure 4. Distribution of max deformation across time

3.3 Frustum Culling Analysis

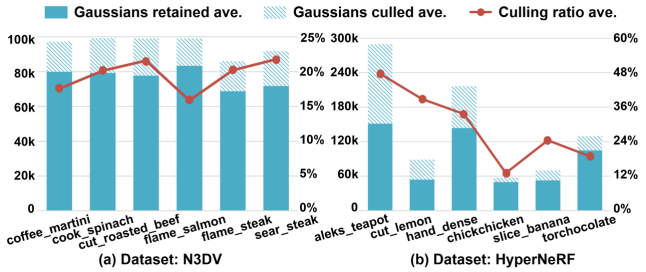

Figure 5. Culling ratio during preprocessing stage

***Observation 3**: Significant waste occurs in deformation of culled Gaussians.* In the standard pipeline, the deformation field computes the exact positions of all Gaussians at the current time step before frustum culling of preprocessing stage, where the invisible ones are excluded throughout the whole procedure afterwards. Fig. 5 indicates that a considerable number and percentage of primitives are culled, making their preceding deformation computations wasteful. Building upon *Observation 2*, we can effectively approximate frustum culling using the canonical, undeformed coordinates to preemptively skip their deformation calculations.

3.4 Tile Workload Imbalance

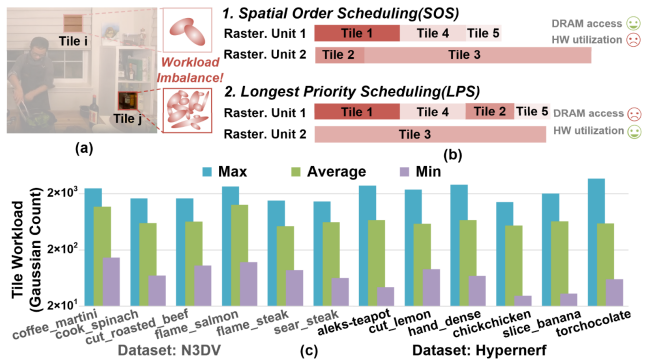

Figure 6. Tile workload statistics

***Observation 4**: Tile workloads are highly skewed, requiring a carefully designed scheduling algorithm to balance hardware utilization with data reuse.* Since rasterization executes independently on each tile, as illustrated in Fig. 6a, variations in scene texture complexity result in highly uneven computational loads. Fig. 6c quantifies this phenomenon by illustrating the workload distribution among tiles. Due to the strict data dependencies inherent in pixel blending, tile workloads cannot be arbitrarily partitioned, which restricts their execution to a single rasterization unit. Furthermore, because most Gaussians span multiple tiles, adjacent tiles frequently access overlapping data sets. This overlap implies that tiles processed sequentially within the same rasterization unit should maintain spatial proximity to enhance memory access reuse. As shown in Fig. 6b, default spatial-ordered scheduling (SOS) arranges tile tasks according to the default z-order and dispatches the foremost tasks to the earliest idle core, susceptible to long-tail effects that diminish hardware utilization and increase overall latency. Conversely, longest-priority scheduling (LPS) sorts tile tasks globally from heaviest to lightest and always assigns the currently heaviest task to an idle core to maximize instantaneous utilization while neglecting data similarity between neighboring tiles, therefore disrupting DRAM access patterns and significantly increasing cache miss rates. The integration of foveated rendering further exacerbates workload imbalance by introducing varying downsampling rates across different regions. Consequently, it is imperative to develop a load-balancing scheduling algorithm that simultaneously optimizes hardware utilization and memory access regularity.

4 Algorithm Optimization

4.1 Motion Tagging and Deformation Skipping (MTDS)

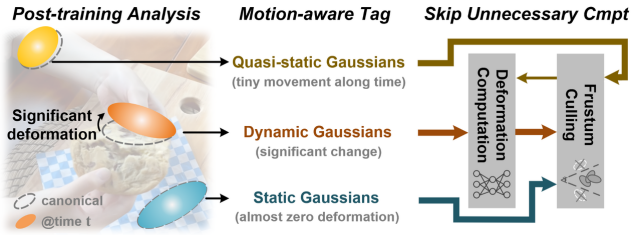

Figure 7. Motion tagging and deformation skipping

Motivated by ***Observation 2/3***, we introduce a post-training tagging methodology that classifies Gaussians into static, quasi-static, and dynamic categories. These assigned tags are directly integrated into the 4D assets. During the subsequent rendering phase, primitives follow distinct data flows based on their categories, effectively bypassing computationally expensive deformation evaluations. The tagging process operates offline following model training by analyzing

and sorting the maximum historical coordinate deformation (called motion) of each Gaussian center across the entire temporal span. The primitives are subsequently partitioned into 3 groups according to predefined empirical ratios. To facilitate regular memory access patterns, primitives of the same category are stored contiguously in main memory. Within each categorical partition, the data is further organized based on spatial proximity to maximize locality. Because each tag requires merely 2 bits, the additional storage overhead is entirely negligible. Following this post-training classification, the rendering pipeline dynamically adjusts its execution flow as Fig. 7. For static Gaussians, deformation computations are strictly bypassed across all time steps. For quasi-static Gaussians, frustum culling precedes the deformation stage, ensuring that the deformation field is only evaluated for the visible primitives. For dynamic Gaussians, the pipeline computes the deformation field prior to evaluating frustum visibility.

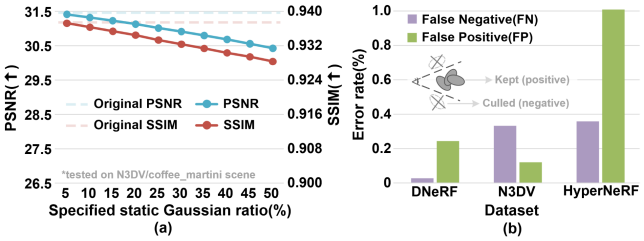

Figure 8. Rendering metrics deterioration of various static ratio specified

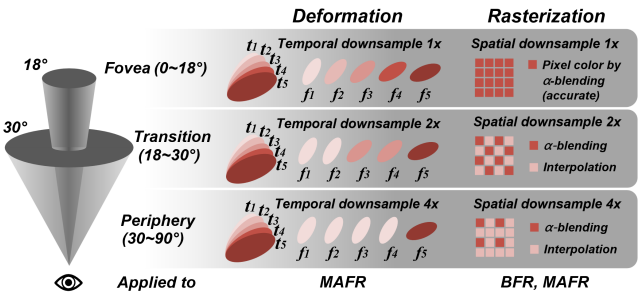

Figure 9. MAFR

Extensive experiments validate the minimal impact of bypassing deformation computations for the static and quasi-static categories respectively. Fig. 8a illustrates that classifying varying proportions of Gaussian primitives introduces only minor degradations in standard rendering metrics (PSNR & SSIM). Furthermore, Fig. 8b demonstrates that designating the 90% of Gaussian primitives with the smallest motion as quasi-static and performing frustum culling prior to deformation yields a tiny erroneous culling rate, which inherently confirms that the impact on overall rendering quality is negligible. Importantly, any culling inaccuracies introduced by this bypass mechanism predominantly manifest near the image boundaries. When integrated with foveated rendering, which naturally degrades the perceived resolution in the peripheral vision, these boundary errors become virtually imperceptible to the human eye. Consequently, the visual approximations introduced by our tagging strategy are highly acceptable.

4.2 Motion-aware Foveated Rendering (MAFR)

To integrate 4DGS with foveated rendering to further accelerate 4DGS inference, we progressively proposed two schemes, Basic Foveated Rendering (BFR) and Motion-aware Foveated Rendering (MAFR). For fundamental BFR, we carefully partition the rendering frame into three distinct regions based on the eccentricity angle from the user gaze point and apply differential downsampling rates to the corresponding tiles during rasterization. For the central foveal region, [11] indicates that an 18° threshold provides an absolutely safe high-resolution zone in VRs when accounting for eye-tracking errors, aligning with the conservative settings of 10° to 15° established by [3]. Regarding the peripheral zone, [17] demonstrate that beyond 30°, detection acuity significantly exceeds resolution acuity. Guided by these physiological insights and existing foveated rendering frameworks such as A3FR[23], we adopt a highly conservative peripheral downsampling strategy. Specifically, we apply downsampling factors of 1x, 2x, and 4x to the central region spanning 0~18°, the transitional region from 18°~30° , and the peripheral region from 30°~90°, respectively.

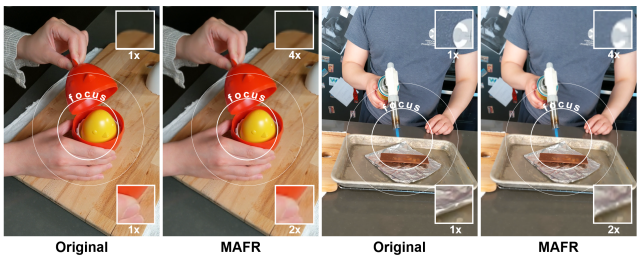

Figure 10. MAFR results

We further extend BFR to the deformation stage of 4DGS to form MAFR, which accounts for the dynamic properties of the scene. Given that human spatial and temporal sensitivity both decrease significantly in the periphery, MAFR downsamples the deformation calculations for Gaussians in these regions to reduce computational overhead. We introduce a lightweight eccentricity calculation in the deformation stage. Each Gaussian is assigned a deformation refresh rate based on its eccentricity range, adhering to the 1×, 2×, and 4× ratios for the central, transition, and peripheral zones. For example, the deformation results for Gaussians in the peripheral zone are updated only once every 4 frames. By

exploiting the inherent limitations of HVS in both spatial and temporal dimensions, MAFR minimizes redundant operations in deformation and rasterization, the two most computationally intensive stages. Furthermore, MAFR maintains seamless compatibility with motion tags. Visual results as Fig. 10 demonstrate that peripheral downsampling is nearly imperceptible when the user focuses on the white circle, i.e. fovea. The effectiveness of this heuristic foveated scheme is quantitatively validated using the HVSQ metric in Sec. 6.2.

5 Architectural Design

5.1 Overall Architecture

To support the optimized algorithm, we design a dedicated hardware architecture and scheduling scheme to further enhance acceleration and energy efficiency. The proposed architecture primarily comprises a unified deformation-preprocess engine (UDPE), a hierarchical sort engine (HSE), a foveated rasterizing engine (FRE), and a workload balancing scheduler (WBS), as illustrated in Fig. 11. Gaussian primitives equipped with motion tags are stored contiguously in DRAM according to their specific motion categories. Within each categorical partition, the primitives are sequentially ordered to maximize spatial locality. The rendering process for each frame initiates upon receiving a camera viewpoint, a gaze direction, and a specific time step. In VR/AR, specialized perception devices typically provide these dynamic parameters at a predefined sampling frequency.

The rendering pipeline begins as the Gaussian primitives enter the UDPE. Based on their assigned categories, these primitives follow distinct data paths to undergo deformation, preprocessing computations, and intersection tests. This stage outputs the updated Gaussian parameters for the current frame and task workload for each image tile. Subsequently, the tile workload lists and the gaze direction are forwarded to the WBS, which executes the WSLAS. The scheduled tile workloads are then dispatched to the HSE, which performs a two-stage depth sorting process adopting the established methodology of GSCore[8]. Following this, the sorted index chunks proceed to the FRE. This engine performs alpha blending by applying specific downsampling rates dictated by the spatial region(e.g. fovea) of the tile. To restore the final image to its predefined resolution, the FRE calculates the colors of the omitted pixels using bilinear interpolation. The overall data flow builds upon mature pipelined tile-based architectures [6, 8, 14, 15] while introducing specialized designs for the UDPE, WBS, and FRE to physically realize the proposed deformation bypassing, workload scheduling, and foveated rendering techniques.

5.2 Workload Balancing Scheduler (WBS)

AS ***Observation 4*** indicates, tilw workload imbalance poses challenges to scheduling algorithms in multi-core systems, especially when foveated rendering introduces varying downsampling across different tiles which further exacerbates workload imbalance. Thus, implementing a scheduling scheme that concurrently optimizes hardware utilization and memory access efficiency is crucial. To address this challenge, we introduce a Windowed Spatial-Locality Aware Scheduling (WSLAS) algorithm. This method maintains global spatial continuity to ensure regular memory access patterns while executing local greedy scheduling to maximize hardware utilization as Fig. 12. Assuming a system comprising M image tiles and N rasterization units, we approximate the computational workload of each tile using the total number of intersecting Gaussian primitives. As detailed in Algorithm 1, the proposed scheduler initially orders the M tiles along a Hilbert curve to construct a continuous task queue. It subsequently deploys a sliding window of capacity K, where K is configured as a multiple of N. The scheduler extracts the initial K tiles from the queue to populate this window. Whenever a rasterization unit becomes available, the scheduler evaluates the undistributed tasks within the active window and greedily assigns the tile with the maximum workload to the idle unit, effectively implementing a longest processing time strategy. Once the leading tile in the window is successfully dispatched, the window advances to incorporate a new tile from the queue into the candidate pool. Pseudo code of WSLAS is as Algo. 1.

Algorithm 1 Windowed Spatial-Locality Aware Scheduling

Phase 1: Spatial Reordering and Initialization
Compute Hilbert curve index $H(t)$ for each tile $t \in \mathcal{T}$
Sort $\mathcal{T}$ in ascending order of $H(t)$
Initialize sliding window $\mathcal{W}$.Append(Dequeue($\mathcal{T}, K$))

Phase 2: Dynamic Scheduling Loop
while $|\mathcal{W}| < K$ **and** $\mathcal{T} \neq \emptyset$ **do**
  $t_{head} \leftarrow$ Head($\mathcal{W}$)
  // Dispatch to Idle Units (Greedy Load Balancing)
  for each unit $r \in \mathcal{R}$ **do**
    if r is IDLE **and** $\mathcal{W} \neq \emptyset$ **then**
      // Select the heaviest tile within the current window
      $t_{heaviest} \leftarrow \arg\max_{t \in \mathcal{W}}(\text{Workload}(t))$
      Dispatch $t_{heaviest}$ to unit r
      $\mathcal{W}$.pop($t_{heaviest}$)
      // Conditional Window Slide & Refill
      if $t_{heaviest} == t_{head}$ **then**
        // If head tile dispatched:
        // Slide forward and refill to capacity K
        $\mathcal{W}$.Append(Dequeue($\mathcal{T}, K - |\mathcal{W}|$))
      end if
    end if
  end for
  Wait for next clock cycle or event
end while

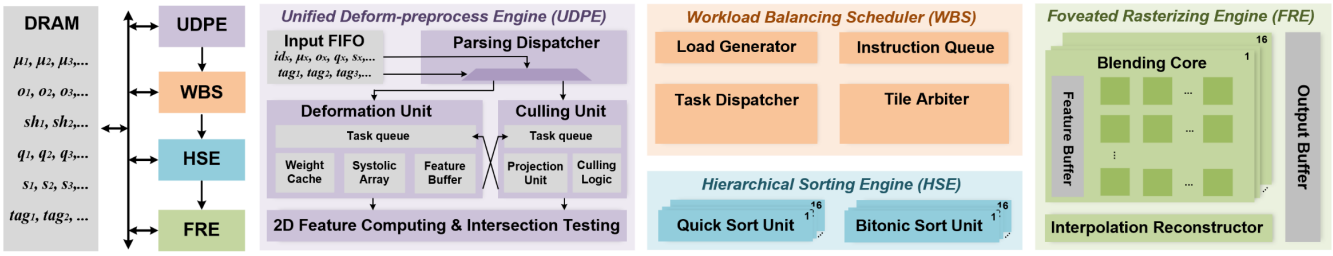

Figure 11. Proposed 4DGS accelerator architecture and module specification

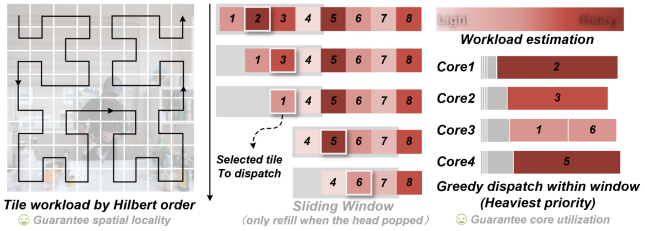

Figure 12. WSLAS

WBS physically implements WSLAS algorithm through 4 core submodules. The task generator receives the preprocessed tile workloads from UDPE, applies specific downsampling rates based on the visual region of each tile, estimates the computational workload utilizing the Gaussian count within the tile, and outputs the tasks to The Instruction Queue (IQ) following a Hilbert curve sequence. IQ stores the information of the currently active K tiles waiting for scheduling in this specific order. To maximize efficiency, the Arbiter utilizes a comparator tree to rapidly identify the tile with the maximum workload from the active window. Finally, the Dispatcher monitors the status of the subcores within HSE and FRE, controls the data path routing, and forwards the selected tile to the HSE for two-stage sorting.

5.3 Unified Deformation-Preprocess Engine (UDPE)

To maximize hardware parallelism, UDPE integrates deformation computation with preprocessing functions including frustum culling, 2D feature computation, and tile intersection testing. Upon entering the engine, a parsing dispatcher first evaluates the lightweight Gaussian tag list to identify the motion category of each primitive. Subsequently, multiplexers and a crossbar network route the corresponding data streams to distinct hardware paths comprising a culling unit and a deformation unit. The deformation unit executes a lightweight neural network to generate the updated parameters for the current frame. Concurrently, the culling unit calculates the camera-space coordinates of the primitives and evaluates their position relative to the viewing frustum to assign visibility flags, which directly dictates the global workload. Once a primitive is confirmed as visible and possesses valid parameters, the pipeline forwards it to the intersection testing unit to compute its screen-space features and identify the affected image tiles. This architectural design strictly preserves data dependencies while minimizing hardware idle time and sustaining continuous on-chip data flow.

5.4 Foveated Rasterizing Engine (FRE)

FRE executes the foveated rasterization process, where the critical operations encompass α blending and bilinear interpolation. This engine consists of 16 Blending Cores (BCs) and a shared interpolation module. Each blending core features an 8x8 array of blending units, and each unit contains 4 parallel lanes responsible for α calculation, transmittance updating with early termination, and color blending of 4 individual pixels. To enhance data retention and preserve strict data dependencies, each BC maintains a dedicated buffer to store Gaussian attributes required for the current tile, which rigidly restricts a single core to processing pixels from only one specific tile. α blending initiates as soon as the corresponding sorting unit finishes processing the first chunk of data. The computed pixel colors are temporarily stored in the buffer for the interpolation module, which performs bilinear interpolation to reconstruct the omitted pixels. This module actively retains the boundary pixels of the currently rendered tile in the cache to eliminate seam artifacts during cross-tile interpolation before writing the final results back to the frame buffer.

6 Evaluation

6.1 Setup

Benchmarks. We evaluate the proposed 4DGS rendering pipeline using widely adopted and complex real-world datasets, N3DV [9] and HyperNeRF [16], which consist of 12 scenes totally. To maintain consistency with the original evaluation metrics and ground truth data, we fix the resolution at 1352×1014 for N3DV and 960×536 for HyperNeRF. All experiments utilize camera poses from the test sets, covering 300 frames per scene. The Gaussian models are trained for 14,000 iterations to ensure both accuracy and convergence.

Baselines. For software-level comparisons, we compare our approach against the original 4DGS[22] and the Neo[15]

software optimization, denoted as Neo-SW, with all implementations executed on Nvidia Jetson AGX Orin. For system-level comparisons, we evaluate three baseline systems including the Jetson AGX Orin 64GB, GSCore[8], and Neo[15]. Since GSCore and Neo are originally designed for 3DGS, we integrate a standard deformation module into these baselines, which is identical to the one in our UDPE, to enable support for the standard 4DGS workflow. To ensure a fair comparison, GSCore, Neo, and our proposed architecture are all configured with 16 sorting and rasterization cores operating at 1GHz. These systems utilize LPDDR4 as the main memory with a consistent bandwidth of 51.2 GB/s.

System configurations. We set the allocation ratios for static, quasi-static, and dynamic motion tags to 40%, 40%, and 20% respectively to achieve a balance between rendering quality and acceleration. Although foveated information is typically provided by external sensors, we conservatively fix the gaze point at the center of the frame to maintain generality. The tile and subtile sizes are configured to 32×32 and 4×4 respectively. Both the HSE and FRE contain 16 cores, and the on-chip SRAM buffer is set to xxx KB. The WBS window size K is set as 32.

Hardware implementation. We synthesize the RTL design of the hardware modules using Synopsys Design Compiler with the TSMC 22nm process at 1GHz to determine the area and power consumption. The area and power breakdowns are illustrated in Fig. Furthermore, we design a cycle-accurate simulator based on discrete-event simulation to obtain precise latency measurements under actual workloads for different scenes, which serves as the basis for evaluating the overall speedup.

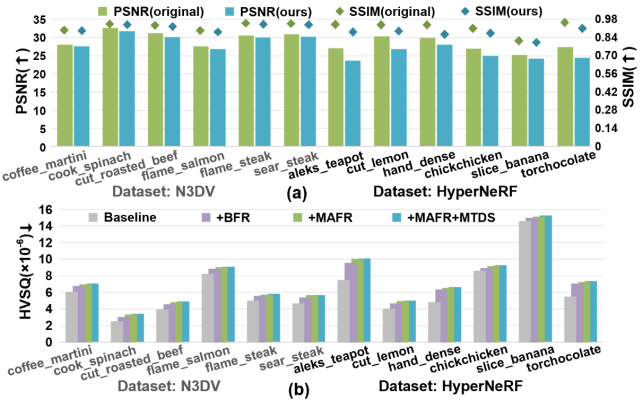

Figure 13. Rendering Quality

6.2 Rendering Quality and Software Speedup

We use 2 typical metrics, PSNR and SSIM to evaluate the impact of Motion-aware Foveated Rendering (MAFR) on rendering quality as Fig. 13a. Our method results in an average SSIM loss of only 0.026 and a PSNR loss of 1.6 dB, which is quite acceptable. For Basic Foveated Rendering (BFR) and Motion-aware Foveated Rendering (MAFR) We utilize HVSQ[10] to measure their perceptual rendering quality for HVS. The HVSQ computation follows the open-source MetaSapiens[10] implementation, and smaller HVSQ values indicate a smaller discrepancy from the ground truth. As Fig. 13b indicates, compared with the original algorithm, the introduction of MAFR does not produce a significant effect on perceived visual quality, and the combined use of MTDS and MAFR likewise yields only a small impact on human visual perception.

We also measure the acceleration achieved by MTDS and MAFR relative to the original algorithm and to Neo's software-optimized variant Neo-SW[15], as Fig. 14. Although software-only speedups are generally modest, our method attains up to 4.2× acceleration in some scenes. This observation suggests that prior optimizations targeting static-scene applications do not achieve optimal performance in dynamic scenes.

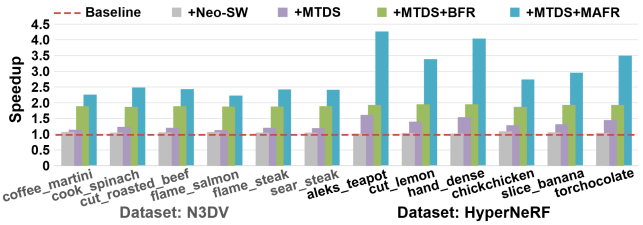

Figure 14. WSLAS

6.3 System performance

To demonstrate the effectiveness of the scheduling scheme WSLAS, we measure rasterization throughput and DRAM conflicts. The baselines include spatial-ordered scheduling (SOS) and longest-priority scheduling (LPS) as introduced in Sec. 3.4. SOS arranges tile tasks according to the spatial order. LPS which always assigns the currently heaviest task greedily, is proven to be the optimal solution for the makespan-minimization problem in multi-machine scheduling when it ignores spatial data locality across tiles. Experimental results as Fig. show that wslas attains a favorable balance between latency and memory accesses.

References

[1] Runze Fan, Jian Wu, Xuehuai Shi, Lizhi Zhao, Qixiang Ma, and Lili Wang. 2025. Fov-GS: Foveated 3D Gaussian Splatting for Dynamic Scenes. *IEEE Transactions on Visualization and Computer Graphics* 31, 5 (May 2025), 2975–2985. doi:10.1109/TVCG.2025.3549576

[2] Linus Franke, Laura Fink, and Marc Stamminger. 2025. VR-Splatting: Foveated Radiance Field Rendering via 3D Gaussian Splatting and Neural Points. *Proc. ACM Comput. Graph. Interact. Tech.* 8, 1 (May 2025), 18:1–18:21. doi:10.1145/3728302

[3] Brian Guenter, Mark Finch, Steven Drucker, Desney Tan, and John Snyder. 2012. Foveated 3D graphics. *ACM Trans. Graph.* 31, 6 (Nov. 2012), 164:1–164:10. doi:10.1145/2366145.2366183

[4] Wei-Hsing Huang, Cheng-Jhih Shih, Jian-Wei Su, Samuel Wade Wang, Vaidehi Garg, Yuyao Kong, Jen-Chun Tien, Nealson Li, Arijit Raychowdhury, Meng-Fan Chang, Yingyan, Lin, and Shimeng Yu. 2025. 3DGauCIM: Accelerating Static/Dynamic 3D Gaussian Splatting via Digital CIM for High Frame Rate Real-Time Edge Rendering. doi:10.48550/arXiv.2507.19133

[5] Yi-Hua Huang, Yang-Tian Sun, Ziyi Yang, Xiaoyang Lyu, Yan-Pei Cao, and Xiaojuan Qi. 2024. SC-GS: Sparse-Controlled Gaussian Splatting for Editable Dynamic Scenes. In *CVPR 2024*. 4220–4230.

[6] Joongho Jo and Jongsun Park. 2025. GS-TG: 3D Gaussian Splatting Accelerator with Tile Grouping for Reducing Redundant Sorting while Preserving Rasterization Efficiency. In *DAC2025*. 1–7. doi:10.1109/DAC63849.2025.11133283

[7] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering.

[8] Junseo Lee, Seokwon Lee, Jungi Lee, Junyong Park, and Jaewoong Sim. 2024. GSCore: Efficient Radiance Field Rendering via Architectural Support for 3D Gaussian Splatting. In *ASPLOS2024 (ASPLOS '24, Vol. 3)*. ASPLOS2024, 497–511. doi:10.1145/3620666.3651385

[9] Tianye Li, Mira Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, Steven Lovegrove, Michael Goesele, Richard Newcombe, et al. 2022. Neural 3d video synthesis from multi-view video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 5521–5531.

[10] Weikai Lin, Yu Feng, and Yuhao Zhu. 2025. Metasapiens: Real-time neural rendering with efficiency-aware pruning and accelerated foveated rendering. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 669–682.

[11] Weikai Lin, Yu Feng, and Yuhao Zhu. 2025. MetaSapiens: Real-Time Neural Rendering with Efficiency-Aware Pruning and Accelerated Foveated Rendering. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*. Association for Computing Machinery, 669–682. doi:10.1145/3669940.3707227

[12] Weikai Lin, Sushant Kondguli, Carl Marshall, and Yuhao Zhu. 2025. PowerGS: Display-Rendering Power Co-Optimization for Neural Rendering in Power-Constrained XR Systems. In *Proceedings of the SIGGRAPH Asia 2025 Conference Papers*. 1–12.

[13] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. 2024. Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis. In *2024 International Conference on 3D Vision (3DV)*. 800–809. doi:10.1109/3DV62453.2024.00044

[14] Cheng Nian, Xiaorui Mo, Jiaying Peng, Weiyi Zhang, Fasih Ud Din Farrukh, Fei Chen, and Chun Zhang. 2025. HyperGS: Efficient Real-Time 3D Gaussian Rendering Processor Through Hierarchical Sorting. In *ISCAS2025*. 1–5. doi:10.1109/ISCAS56072.2025.11044261

[15] Changhun Oh, Seongryong Oh, Jinwoo Hwang, Yoonsung Kim, Hardik Sharma, and Jongse Park. 2025. Neo: Real-Time On-Device 3D Gaussian Splatting with Reuse-and-Update Sorting Acceleration. ASPLOS2026. doi:10.48550/arXiv.2511.12930

[16] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T Barron, Sofien Bouaziz, Dan B Goldman, Ricardo Martin-Brualla, and Steven M Seitz. 2021. Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields. *arXiv preprint arXiv:2106.13228* (2021).

[17] Anjul Patney, Marco Salvi, Joohwan Kim, Anton Kaplanyan, Chris Wyman, Nir Benty, David Luebke, and Aaron Lefohn. 2016. Towards foveated rendering for gaze-tracked virtual reality. *ACM Trans. Graph.* 35, 6 (Dec. 2016), 179:1–179:12. doi:10.1145/2980179.2980246

[18] Minnan Pei, Gang Li, Junwen Si, Zeyu Zhu, Zitao Mo, Peisong Wang, Zhuoran Song, Xiaoyao Liang, and Jian Cheng. 2025. GCC: A 3DGS Inference Architecture with Gaussian-Wise and Cross-Stage Conditional Processing. In *MICRO2025 (MICRO '25)*. 1824–1837. doi:10.1145/3725843.3756072

[19] Jiawei Ren, Liang Pan, Jiaxiang Tang, Chi Zhang, Ang Cao, Gang Zeng, and Ziwei Liu. 2023. DreamGaussian4D: Generative 4D Gaussian Splatting.

[20] Yiyang Sun, Qinzhe Zhi, Yiqi Jing, Le Ye, Ru Huang, and Tianyu Jia. 2025. Local-GS: An Order-Independent Gaussian Splatting Training Accelerator Exploiting Splat Locality. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. 1–7. doi:10.1109/DAC63849.2025.11132792

[21] Linye Wei, Jiajun Tang, Fan Fei, Boxin Shi, Runsheng Wang, and Meng Li. 2025. No Redundancy, No Stall: Lightweight Streaming 3D Gaussian Splatting for Real-time Rendering. *ICCAD2025* (Nov. 2025).

[22] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 2024. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *CVPR 2024*. IEEE, Seattle, WA, USA, 20310–20320. doi:10.1109/CVPR52733.2024.01920

[23] Shuo Xin, Haiyu Wang, and Sai Qian Zhang. 2025. A3FR: Agile 3D Gaussian Splatting with Incremental Gaze Tracked Foveated Rendering in Virtual Reality. In *Proceedings of the 39th ACM International Conference on Supercomputing*. 279–292. doi:10.1145/3721145.3735112

[24] Mengtian Yang, Yipeng Wang, Chieh-Pu Lo, Xiuhao Zhang, Sirish Oruganti, and Jaydeep P. Kulkarni. 2025. GSAcc: Accelerate 3D Gaussian Splatting via Depth Speculation and Gaussian-centric Rasterization. In *DAC2025*. 1–7. doi:10.1109/DAC63849.2025.11133032

[25] Xiaolong Yang, Yang Wang, Wende Xu, Yubin Qin, Huanyu Wang, Ruiqi Guo, Zhiheng Yue, Jiangyuan Gu, Shaojun Wei, Yang Hu, and Shouyi Yin. 2026. A 0.24mJ/Frame Quadratic Interpolation 4DGS Processor with Recursive Computation Reuse and Tree-Based Parallel-Rendering. In *ISSCC2026*.

[26] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. 2024. Deformable 3D Gaussians for High-Fidelity Monocular Dynamic Scene Reconstruction. In *CVPR 2024*. 20331–20341. doi:10.1109/CVPR52733.2024.01922

[27] Zeyu Yang, Hongye Yang, Zijie Pan, and Li Zhang. 2024. Real-time Photorealistic Dynamic Scene Representation and Rendering with 4D Gaussian Splatting. In *ICLR 2024*.

[28] Chenqi Zhang, Yu Feng, Jieru Zhao, Guangda Liu, Wenchao Ding, Chentao Wu, and Minyi Guo. 2025. STREAMINGGS: Voxel-Based Streaming 3D Gaussian Splatting with Memory Optimization and Architectural Support. In *DAC2025*. 1–7. doi:10.1109/DAC63849.2025.11132470